

ASHEN: Action-Aware Retrieval-Augmented LLM Guidance for Authorized Penetration Testing with Closed-Loop Remediation Validation

[PLACEHOLDER: author name]

[PLACEHOLDER: affiliation, email]

Abstract—Large language models (LLMs) are increasingly proposed to assist penetration testing, but most systems stop at recommendation and are evaluated subjectively. We present ASHEN, an AI-assisted penetration-testing platform that closes the full loop — scan, vulnerability detection, exploit validation, retrieval-augmented attack recommendation, remediation guidance, and *re-validation of the generated fix by re-exploitation* — under an explicit authorization-and-audit governance model, using a *local* LLM so that no target data leaves the operator’s network. We make three contributions. First, an evaluation harness that uses ASHEN’s own exploit runner as an *objective oracle*: an exploit type is “correct” for a target iff running it returns a positive verdict, enabling scoring of recommendation ranking and remediation efficacy without human judgement. Second, using this oracle we show that *naively* injecting retrieved CVE knowledge into the recommender degrades action selection, and that an *action-aware grounding* scheme — mapping retrieved CVEs onto the tool’s available exploit types — restores correct ranking while keeping the recommendation grounded. Third, we define *remediation efficacy* as the fraction of confirmed vulnerabilities that no longer validate after the generated fix is applied. In a feasibility pilot (4 remediation trials, efficacy 0.25), the loop caught three confident-but-incorrect LLM fixes — including one whose command *executed successfully and passed the LLM’s own validation step* yet left the host exploitable — that a recommendation-only, human-rated, or self-validating evaluation would have accepted. Applying the same grounding scheme to the remediation step then recovers the failures end-to-end (efficacy 0.25→0.75), unifying our two results: grounding corrects both recommendation and remediation. We describe the protocol for scaling to a larger evaluation.

I. INTRODUCTION

Penetration testing is a manual, expertise-intensive process, motivating a substantial body of work on *autonomous penetration testing* [7]. Two families of decision-making methods dominate: reinforcement-learning (RL) agents that learn attack and path-planning policies in simulated networks [7], [8], and, more recently, large language model (LLM) agents that exploit the models’ contextual semantic understanding to drive the workflow [1], [9]. ASHEN belongs to the LLM-based family; existing LLM-based pentest systems, however, share three weaknesses that limit their scientific standing:

- 1) **They stop at recommendation.** Whether a suggestion *works*, and whether any proposed fix actually *removes* the weakness, is unmeasured.
- 2) **They are evaluated subjectively**, by human raters or anecdotes, so results are hard to reproduce and easy to contest.

- 3) **They send sensitive data to a third-party API** — target IPs, detected vulnerabilities, credentials — a confidentiality problem for real engagements.

ASHEN addresses all three. It runs an *on-premise* LLM (no data exfiltration), it *validates* every exploit it recommends and every fix it proposes against the live target, and it is wrapped in an authorization-gated governance model with dual audit logs. Because ASHEN can *execute* exploits, the same execution serves as an *objective oracle* for evaluation.

Contributions.

- **C1 — An oracle-backed evaluation harness** (§IV) that scores recommendation ranking and remediation efficacy using the exploit runner’s verdict as ground truth, with no human labelling.
- **C2 — Action-aware retrieval grounding** (§III, §V): dumping retrieved CVEs into the prompt biases the model toward CVE-shaped actions the tool cannot perform; mapping retrieved CVEs onto available exploit types fixes this. (*E1, n=3 hosts: robust across Linux, Windows, and Shellshock targets where plain RAG is not; more targets [PLACEHOLDER: at scale].*)
- **C3 — Closed-loop remediation efficacy** (§III, §VI): an end-to-end metric that re-exploits the target after the generated fix is applied. (*Pilot, n=4, efficacy 0.25→0.75 grounded: the loop caught three incorrect fixes — including one that executed successfully and passed its own validation yet left the host exploitable; full study [PLACEHOLDER: more trials].*)

II. BACKGROUND AND RELATED WORK

Autonomous and automated penetration testing. A large literature casts penetration testing as a sequential decision-making problem and applies reinforcement learning (RL) to learn attack and path-planning policies in simulated networks. Liu et al. [7] review RL-based decision-making for autonomous pentesting, and unified modeling frameworks such as AutoPT-Sim [8] standardise the simulation environments these agents train in. Such methods excel at policy learning but operate in simulation, produce no human-readable guidance, and address neither remediation nor data confidentiality — gaps that motivate ASHEN’s LLM-based, on-premise design.

LLMs for offensive security. The closest system is Pen-testGPT [1], which integrates GPT-3.5/GPT-4 and organises a pentest around a *Pentesting Task Tree* driven by three

modules (Reasoning, Generation, Parsing). It is evaluated on 13 HackTheBox/VulnHub targets (182 sub-tasks) and reports a 228.6% task-completion increase over its GPT-3.5 baseline. Two properties separate it from ASHEN: it is cloud/API-backed (target data leaves the network), and it is human-in-the-loop and *offensive-only* — the operator executes the suggested commands and feeds results back, with no remediation-generation or fix-validation component. A parallel line shows LLM agents can autonomously *exploit*: Fang et al. demonstrate autonomous website hacking [3], exploitation of real one-day CVEs with a GPT-4 agent [4], and teams of agents exploiting zero-day vulnerabilities [5]; Happe and Cito study autonomous Linux privilege escalation [6]. Most directly comparable, HackSynth [9] pairs a dual-module LLM agent (Planner/Summarizer) with a CTF-based *evaluation framework* and, like ASHEN, stresses robust safeguards. These drive offence but stop at exploitation, judge success by human or capture-the-flag task-completion rather than an execution oracle, and do not generate or re-validate remediations. We compare capabilities against PentestGPT’s published design (Table I) rather than re-running it (its native backend is a paid GPT-4 API; a fair empirical head-to-head is future work, §VII).

TABLE I
ASHEN VS PENTESTGPT (CAPABILITIES; PENTESTGPT FACTS FROM [1]).

Capability	PentestGPT	ASHEN
LLM backend	GPT-3.5/4 (cloud)	local (llama3.2)
Data exposure	sent to OpenAI	on-premise
Execution	human-in-the-loop	automated
Validation	none	execution-as-oracle
Grounding	task-tree context	RAG + action-aware
Remediation	none	RAG-grounded
Fix re-validation	none	re-exploit loop

Retrieval-augmented generation (RAG). RAG [2] grounds LLM output in a retrieved corpus to reduce hallucination. We show (§V) that in a *tool-using* setting, naive RAG can instead *mis-direct action selection* when the knowledge representation (CVEs) is misaligned with the action space (exploit types).

LLM-assisted remediation. LLMs have been applied to vulnerability repair, e.g. Pearce et al.’s zero-shot vulnerability repair [11], which generates code fixes evaluated against a test suite rather than by re-exploiting a live target. Closest to ASHEN is PenHeal [10], a two-stage framework pairing a pentest module with a remediation module; it *recommends and scores* fixes but does **not** re-validate them by re-exploiting the patched system — precisely the gap ASHEN’s closed loop fills. To our knowledge ASHEN is the first to score remediation by re-exploitation.

Position of ASHEN. ASHEN is distinguished by the combination of (i) local LLM, (ii) execution-as-oracle evaluation, and (iii) closed-loop remediation validation, under (iv) an explicit authorization/audit governance model.

III. ASHEN SYSTEM DESIGN

A. Overview and roles

ASHEN separates an *Analyst* (runs scans/exploits on authorised targets, sees only their own work) from an *Admin* (whitelists target IPs, approves target requests, reads the audit log). The assessment workflow is: *Scan* (Nmap [12] `-sV --script vuln`) → *Vulnerability* (an unvalidated NSE detection) → *Exploit Run* (validates or refutes it) → *Attack Recommendation* (RAG-grounded LLM guidance) → *Remediation* (LLM fix guidance) → *Report*.

B. Exploit types as a typed action space

ASHEN exposes a small, typed set of *exploit types*, each a uniform adapter producing a single verdict envelope (ran, vulnerable, evidence). The current set is four: `ssh_brute_force`, `ftp_brute_force` (credential family) and `ms17_010_check`, `shellshock_cgi` (check family). This typed action space is what makes both the oracle (§IV) and action-aware grounding possible.

C. Retrieval grounding (and its failure mode)

The recommender retrieves CVE context from a vector store (ChromaDB [14] over sentence-transformer embeddings [13]) and conditions a recommendation on it. We compare three grounding conditions: *off* (no retrieval); *plain* (retrieved CVEs concatenated into the prompt — the baseline); and *action-aware* (each retrieved CVE is mapped onto the exploit types whose declared `service + signatures` it matches; CVEs matching no available exploit are surfaced as informational only). The corpus stays tool-agnostic; adding an exploit type changes only that exploit’s declaration, not the corpus.

D. Closed-loop remediation

For a confirmed-vulnerable finding, ASHEN generates a five-part remediation (Root Cause / Immediate Containment / Permanent Fix / Validation / Hardening). The fix is applied to the target, after which the *same exploit is re-run*: the vulnerability is “remediated” iff it no longer validates *and* the service is still reachable.

E. Governance and the local-LLM choice

Every operator action is recorded in an Audit Log; every LLM prompt/response in a separate AI Governance Log. The LLM runs on-premise, so target data never leaves the network — a deliberate confidentiality choice that distinguishes ASHEN from API-backed tools.

IV. EVALUATION METHODOLOGY

A. Execution as an objective oracle

For a target and exploit type, the exploit runner returns a boolean `vulnerable` verdict, treated as *ground truth*: an exploit type is *relevant* iff it returns `vulnerable=True`. This removes human judgement from scoring (a) whether a recommendation ranks working exploits first, and (b) whether a remediation removed the weakness.

TABLE II

GROUNDING CONDITIONS ACROSS THREE HOSTS / ALL FOUR EXPLOIT TYPES (LIVE, SEED $\tau=0$, $N=3$). AGGREGATE P@1: OFF = 1.00, PLAIN = 0.67, ACTION-AWARE = 1.00.

Target	Grounding	Top-1	P@1	Fabr.
MS2 (Lin)	off	ftp_brute_force ✓	1.00	—
MS2 (Lin)	plain	ms17_010_check ✗	0.00	0.60
MS2 (Lin)	action-aware	ssh_brute_force ✓	1.00	1.00*
MS3 (Win)	off	ms17_010_check ✓	1.00	—
MS3 (Win)	plain	ms17_010_check ✓	1.00	—
MS3 (Win)	action-aware	ms17_010_check ✓	1.00	—
Shellshock	off	shellshock_cgi ✓	1.00	—
Shellshock	plain	shellshock_cgi ✓	1.00	0.60
Shellshock	action-aware	shellshock_cgi ✓	1.00	0.50

B. Metrics

Precision@1 / MRR of the recommended exploitation order against the relevant set. **Fabrication rate**: fraction of cited CVEs not in a curated set of CVEs real-and-applicable to the target. **Remediation efficacy**: fraction of confirmed-vulnerable findings that, after the generated fix is applied, no longer validate *and* leave the service up (fixed vs not_fixed vs broke_service).

C. Harness and reproducibility

The harness drives ASHEN’s services directly (no web/auth/DB layer). Generation is seeded (temperature=0, seed=42). A full live run is split into isolated phases (oracle / recommend / remediate) so memory-heavy components do not run concurrently.

D. Testbed

(Pilot, 3 hosts.) Three targets spanning all four exploit types with positive cases: Metasploitable 2 (Linux; vsftpd 2.3.4 backdoor [17], OpenSSH 4.7p1, Apache 2.2.8, Samba 3.x $\rightarrow$ ssh/ftp brute-force), Metasploitable 3 Windows Server 2008 R2 (SMBv1 $\rightarrow$ MS17-010/EternalBlue [15]), and an Ubuntu 14.04 host (Apache mod_cgi + unpatched bash $\rightarrow$ Shellshock [16]). Further scale-up protocol in §VII.

V. E1: GROUNDING AND RECOMMENDATION QUALITY

Live, $n=3$ hosts (all four exploit types), pilot. We use three hosts: Metasploitable 2 (Linux; relevant = {ftp_brute_force, ssh_brute_force}), Metasploitable 3 Windows Server 2008 R2 (relevant = {ms17_010_check}, EternalBlue), and an Ubuntu 14.04 Shellshock host (relevant = {shellshock_cgi}). Ground truth is set per target by the oracle; fabrication is scored against a *per-target* set of applicable CVEs, so EternalBlue counts as valid on Windows but as a fabrication on Linux.

Findings.

- 1) **Plain RAG is unreliable; its failure is target-dependent.** Plain retrieval biases the model toward SMB/CVE-heavy exploits regardless of applicability — *wrong* on the Linux host (ranks the dead-end ms17_010 first, P@1 = 0) but *coincidentally right*

TABLE III

REMEDATION EFFICACY ACROSS THREE HOSTS (LIVE, $N=4$ PILOT). **EFFICACY = 0.25 (1/4 FIXED).**

Host / finding	ASHEN’s prescribed fix	After	Outcome
MS2	rotate weak creds	no creds	fixed
ssh_brute			
MS2	rotate creds (+ bogus allow_root)	ftp:<any> logs in	not_fixed
ftp_brute			
MS3	reg add ...	still	not_fixed
ms17_010	DisableSMBv1=1 (succeeds, key present)	vulnerable after reboot	
Shellshock	update mod_cgi component)	Apache (wrong) still vulner-able	not_fixed

on the Windows host (where EternalBlue truly is the answer, P@1 = 1; Shellshock is correctly ranked too). Aggregated it scores only P@1 = 0.67 — still the worst condition.

- 2) **Action-aware grounding is robust** (P@1 = 1.0 across all three hosts). Mapping retrieved CVEs onto available exploit types and quarantining unmatched ones keeps the recommendation correct *and* grounded on every host — the only condition achieving both; plain sacrifices ranking, *off* sacrifices grounding (it cites nothing).
- 3) **The per-target CVE distinction validates the design.** The same CVE (CVE-2017-0144) is a fabrication on Linux and valid evidence on Windows, because correctness is judged against each host’s actual exposure.
- 4) **A residual fabrication artifact (*, MS2).** Action-aware still cited one off-platform CVE for a low-priority exploit, motivating platform-applicability filtering (§VIII).

Interpretation. The mechanism is a *knowledge-action-space misalignment*: grounding a tool-using LLM on a CVE corpus biases it toward CVE-shaped actions, including ones outside the tool’s capability or inapplicable to the host. Aligning the knowledge to the action space, and judging it per-target, resolves this. A 5-seed $\times$ 2-temperature variance study (§VII) confirms this is not a single-seed artifact: at a task-appropriate temperature action-aware is perfectly stable (1.00 ± 0.00) and plain RAG is the worst at every temperature. *Caveat: $n=3$ hosts.*

VI. HEADLINE: REMEDIATION EFFICACY

Live, $n=4$ trials across 3 hosts, pilot. Four confirmed findings were remediated end-to-end: ASHEN generated its guidance, *its prescribed fix was applied verbatim* to the live host (snapshotted first), and the same exploit was re-run.

Findings.

- 1) **The loop caught a plausible-but-wrong fix (MS2 FTP).** ASHEN asserted weak credentials; re-exploitation showed the real cause is *anonymous login* (the ftp account accepts any password), which credential rotation cannot address.

TABLE IV
UNGROUNDED VS GROUNDED REMEDIATION, RE-VALIDATED
END-TO-END. EFFICACY 0.25 $\rightarrow$ 0.75.

Finding	Ungrounded	Grounded
MS2 ssh_brute	rotate creds: fixed	rotate creds: fixed
MS2 ftp_brute	allow_root: not_fixed	anon..=NO: not_fixed
MS3 ms17_010	DisableSMBv1: not_fixed	SMB1=0: fixed
Shellshock	update Apache: not_fixed	disable CGI: fixed

- 2) **It caught a fix that executed successfully but did nothing (MS3 MS17-010).** ASHEN diagnosed the root cause correctly (SMBv1) but prescribed a non-existent registry value (DisableSMBv1; the effective control is SMB1=0). The command returns “completed successfully,” the key is verifiably present, *and ASHEN’s own suggested validation (a reg query) passes* — yet after reboot the host is still EternalBlue-vulnerable. Every check short of re-attacking reports success; only re-exploitation reveals the fix is hollow.
- 3) **All three ungrounded failures misdiagnosed the fix** — allow_root (non-existent), DisableSMBv1 (wrong registry value), and “update Apache” for Shellshock (wrong component: it is a *bash* bug) — linking to §V.
- 4) **Where diagnosis and command were correct, the fix worked (MS2 SSH)** — the positive control.

Interpretation. The headline metric exposes what recommendation-only, human-rated, *or even self-validating* LLM remediation would miss: a fix’s confidence, structure, successful execution, and passing its own validation step are all uncorrelated with whether the vulnerability is gone. Only execution-based re-validation reveals the difference. *Caveat: n=4 trials across three hosts; not a rate estimate.*

Grounding the remediation closes the gap (re-validated end-to-end). The remediation path, unlike the recommender (§V), was originally *ungrounded* free-form generation — the source of the hallucinated fixes. We applied the same grounding principle (retrieve curated, verified fix references; constrain the LLM to them) and re-ran all four remediations end-to-end (generate $\rightarrow$ apply verbatim $\rightarrow$ re-exploit).

Grounding *corrected the prescription on all four findings* (SSH creds, SMB1=0, anonymous_enable=NO for FTP replacing the invented allow_root, and disabling mod_cgi for Shellshock); three of four then re-validate as fixed. The remaining FTP failure is now isolated to a *deployment-execution* problem — MS2’s inetd-launched vsftpd does not honour anonymous_enable=NO on restart — and is no longer a prescription error: the loop confirms the fix is correct yet ineffective on that host, which is itself the argument for execution-based re-validation. This unifies the paper’s two results: the same grounding principle corrects both recommendation and remediation.

Complete exploit-type coverage. A Shellshock target (Ubuntu 14.04, Apache mod_cgi, unpatched bash) was added so the fourth exploit type — previously only ever a true-negative — is exercised end-to-end. The oracle con-

firmed Shellshock RCE; the grounded remediation correctly prescribed disabling mod_cgi (and patching bash); applying it (a2dismod cgi cgid) re-validated as **fixed** (injection no longer executes, web service up). Across all four exploit types, grounded remediation efficacy is **0.75 (3/4)** — ssh_brute_force, ms17_010_check, shellshock_cgi fixed; ftp_brute_force not_fixed (the deployment-execution case).

VII. VARIANCE AND REMAINING EVALUATION

Seed/temperature variance (live). To check the deterministic E1 result (§V) is not a single-seed artifact, we re-ran all conditions across 5 seeds and 3 targets at two temperatures. Exploit selection is a *decision* task, so a low temperature ($\tau=0.3$) is appropriate; we also stress-test at $\tau=0.7$.

TABLE V
PRECISION@1 VARIANCE (MEAN $\pm$ STDEV, 5 SEEDS $\times$ 3 TARGETS).

Condition	P@1 @ $\tau=0.3$	P@1 @ $\tau=0.7$
off	1.00 $\pm$ 0.00	0.93 $\pm$ 0.26
plain	0.67 $\pm$ 0.47	0.50 $\pm$ 0.50
action-aware	1.00 $\pm$ 0.00	0.79 $\pm$ 0.41

Plain RAG is the worst at *both* temperatures (a robust negative result). At the task-appropriate temperature action-aware is perfectly stable (1.00 $\pm$ 0.00), matching the ungrounded baseline while keeping its CVE justification; its $\tau=0.7$ dip is a diagnosed high-temperature artifact (the model occasionally brute-forcing open ports against the grounding), still well above plain RAG. Fabrication is high/noisy for both RAG conditions (~ 0.6 – 0.7) and does not discriminate them, so we de-emphasise it.

Remaining. [PLACEHOLDER: E2 — ranking across many targets with confidence intervals.] [PLACEHOLDER: Headline at scale: remediation efficacy across $N \approx 20$ – 30 findings, on a testbed adding Metasploitable 3, VulnHub images, a Windows MS17-010 host, and a Shellshock CGI host to exercise all four exploit types with positive cases.] [PLACEHOLDER: Model-strength comparison: weak (tinyllama) vs default (llama3.2) vs strong (GPT-4 via API) to separate “small model is weak” from “grounding scheme”.] PentestGPT: a capability comparison is given in §2 (Table I); a same-target empirical head-to-head (real PentestGPT-GPT-4) is future work — its paid GPT-4 backend and GPT-4-tuned parser make a fair local run impractical.

VIII. DISCUSSION

The headline conceptual result is the *knowledge-action-space misalignment* in tool-using RAG (§V): grounding helps factual accuracy but can hurt *action selection* when retrieved knowledge references capabilities the tool lacks. Action-aware grounding is a general remedy. A residual refinement is *platform-applicability filtering*: do not attach a CVE as evidence for an exploit when the CVE’s product/platform contradicts the target fingerprint.

IX. LIMITATIONS

Few targets (n=3 hosts for E1, n=4 trials for remediation): pilot numbers are not yet significant; §VII defines scale-up. *Four exploit types*: shallow ranking depth; power must come from many targets. *Manual remediation*: ASHEN generates guidance; applying the fix is a human (or operator-authorised) step. *Weak default model*: llama3.2-3B is small.

X. ETHICS AND RESPONSIBLE USE

ASHEN operates only against IPs an Admin has explicitly authorised; Analysts cannot act outside scope. All actions and LLM interactions are audit-logged. The local-LLM design keeps target data on-premise. All experiments were conducted against a self-hosted, intentionally-vulnerable VM owned by the authors on an isolated network.

XI. CONCLUSION

ASHEN closes the penetration-testing loop from detection through validated remediation, using execution as an objective oracle and a local LLM for confidentiality. A three-host pilot demonstrates that action-aware retrieval grounding corrects a failure mode of naive RAG in tool-using agents, and that closed-loop re-validation catches confident-but-incorrect remediations. The multi-target scale-up is in progress.

REFERENCES

- [1] G. Deng *et al.*, “PentestGPT: An LLM-empowered Automatic Penetration Testing Tool,” *USENIX Security*, 2024. arXiv:2308.06782.
- [2] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.
- [3] R. Fang *et al.*, “LLM Agents can Autonomously Hack Websites,” 2024. arXiv:2402.06664.
- [4] R. Fang *et al.*, “LLM Agents can Autonomously Exploit One-day Vulnerabilities,” 2024. arXiv:2404.08144.
- [5] R. Fang *et al.*, “Teams of LLM Agents can Exploit Zero-Day Vulnerabilities,” 2024. arXiv:2406.01637.
- [6] A. Happe and J. Cito, “LLMs as Hackers: Autonomous Linux Privilege Escalation Attacks,” 2023. arXiv:2310.11409.
- [7] J. Liu, Y. Zhang, S. Zhou, J. Yang, Y. Lu, and X. Zhong, “Autonomous Penetration Testing using Reinforcement Learning: A Review and Perspectives,” *Expert Systems with Applications*, 2025.
- [8] Y. Wang *et al.*, “A Unified Modeling Framework for Automated Penetration Testing (AutoPT-Sim),” 2025. arXiv:2502.11588.
- [9] L. Muzsai, D. Imolai, and A. Lukács, “HackSynth: LLM Agent and Evaluation Framework for Autonomous Penetration Testing,” 2024. arXiv:2412.01778.
- [10] J. Huang and Q. Zhu, “PenHeal: A Two-Stage LLM Framework for Automated Pentesting and Optimal Remediation,” *Workshop on Autonomous Cybersecurity*, 2024. arXiv:2407.17788.
- [11] H. Pearce *et al.*, “Examining Zero-Shot Vulnerability Repair with Large Language Models,” *IEEE S&P*, 2023.
- [12] G. F. Lyon, *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*. Insecure.Com LLC, 2009.
- [13] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *EMNLP*, 2019.
- [14] Chroma, “Chroma: the open-source embedding database,” <https://www.trychroma.com>, 2023.
- [15] Microsoft, “Security Bulletin MS17-010: Security Update for Microsoft Windows SMB Server,” 2017. CVE-2017-0143/0144/0145.
- [16] NVD, “CVE-2014-6271 (Shellshock): GNU Bash environment variable command injection,” 2014.
- [17] NVD, “CVE-2011-2523: vsftpd 2.3.4 backdoor command execution,” 2011.